

# 全体像

## 目次

|                      |   |
|----------------------|---|
| 1 既存ドキュメントとの関係 ..... | 9 |
|----------------------|---|

## i この文書の位置づけ

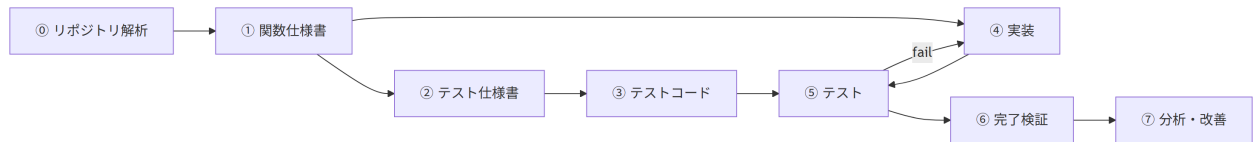
本サイトは legacy-reverse の**設計・仕様**を扱う。「どう使うか」は既存の利用者向けドキュメント (スライド版チュートリアル・QUICKREF・MANUAL の HTML/PDF) が担い、本サイトは「なぜこの構造か・各部品は何を保証するか・どう運用するか」を、このページ(全体)→各章(詳細)の順で読めるようにしたもの。

何を解決するシステムか

レガシーコード(Fortran / C# 等)を Python へ仕様ベースで移植するための Claude Code スキル群+決定的スクリプト群+MCP サーバ。「レガシーを読んで直接書き写す」のではなく、関数仕様書という中間成果物を挟み、独立に作ったテストと実装を突き合わせることで品質を機械的に担保する。

パイプラインは8 フェーズ。最初のトリガは必ず人が引く(AI が勝手にフェーズを進めることはない)。連続実行を開始した場合だけ、その1回のトリガのあと工程間の遷移が自動になる — ただし承認・裁定は必ず人を待つ。

図 1



要点は①仕様書だけを共通の親とする2系統(テスト系②③、実装系④)を⑤で突き合わせること。②③④は互いを見ず、レガシー原文も見ない。仕様書の曖昧さ・穴は、必ずテスト失敗または spec-gap ISSUE として表面化する。

設計原則(5つ)

表 1

| 原則            | 内容                                                        | それで防ぐもの              |
|---------------|-----------------------------------------------------------|----------------------|
| クリーンルーム分業     | ②③はレガシー原文を見ない。③は②だけ、④は①だけを入力にする                           | 「レガシーの写経」による仕様の暗黙化   |
| 状態はすべてファイル    | 進捗の正は functions.json・ledger.json・成果物フロントマター。会話コンテキストに置かない | セッション切れによる「1 からやり直し」 |
| 機械が正、AI は意味づけ | 列挙・検証・台帳操作は決定的スクリプト。AI は読解と執筆だけ                           | ハルシネーション・省略・数え漏れ     |
| 人の承認ゲート       | 仕様確定・テスト承認・裁定は人。AI は「仮説+Yes/No」で聞く                        | AI による勝手な仕様確定        |
| 強制は仕組みで       | テスト保護は hook、スタブ検知・引用検証はスクリプト                              | 「プロンプトで禁止」の不確実さ      |

詳細: アーキテクチャと設計判断

## システムの 3+1 層

図 2

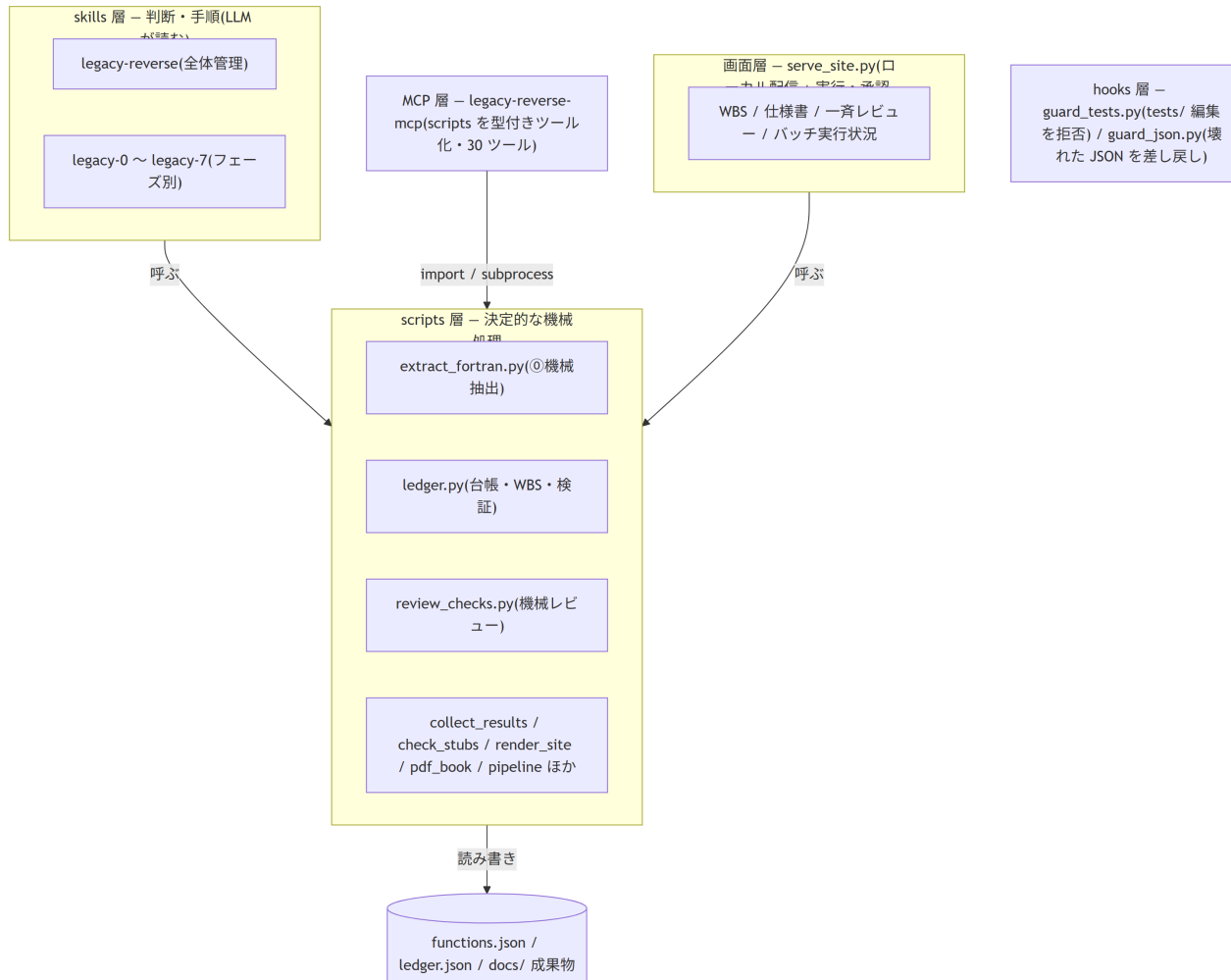


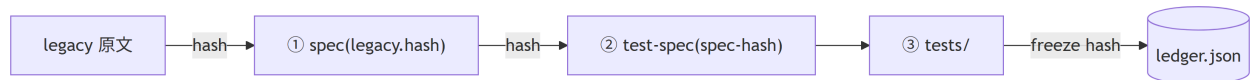
表 2

| 層       | 役割                                                                | 章                  |
|---------|-------------------------------------------------------------------|--------------------|
| skills  | 「いつ・何を・どの順で」を書いた手順書。レガシー読解・仕様執筆・トリアージの <b>判断だけ</b> を LLM が行う      | アーキテクチャ / パイプライン仕様 |
| scripts | 単体でも動く決定的処理。列挙・検証・台帳・レンダリングの <b>正</b>                             | 品質ゲート仕様 / データ仕様    |
| MCP     | scripts の型付きラップ(判断は持たない・二重管理なし)。許可プロンプトとシェル事故を減らす                 | MCP サーバ仕様          |
| hooks   | LLM の自制に頼らない強制層。PreToolUse は <b>拒否</b> 、PostToolUse は <b>差し戻し</b> | 品質ゲート仕様            |
| 画面      | 人の 4 つの仕事(トリガ・回答・承認・裁定)をブラウザで完結させる操作面                             | 画面設計               |

## 整合性の背骨: ハッシュ連鎖

①→②→③ の各成果物は上流のハッシュを記録する。上流が改訂されると下流の照合が落ち、WBS に stale△ / △改変が自動表示される。「①を直したのに②を直し忘れた」は構造的に起きない。

図 3



詳細: データ仕様 — ハッシュ連鎖と status 遷移

品質保証の分担(全体)

LLM 成果物は人に届く前に、決定的スクリプトの検証を必ず通る。

表 3

| 誤りの種類              | 検知する仕組み                                |
|--------------------|----------------------------------------|
| 数え漏れ(⑥)            | 抽出器内蔵の 2 系統カウント突合                      |
| 実在しない根拠の引用・省略(①)   | review_checks.py spec                  |
| 仕様 ID の捏造・ケース漏れ(②) | review_checks.py testspec              |
| ケースとテスト関数の過不足(③)   | マーカー突合(collect_results exit 3)         |
| スタブ実装(④)           | check_stubs.py                         |
| 上流改訂の見落とし(⑤前)      | ledger verify                          |
| 意味レベルのすり替え         | <b>機械では取れない</b> → 人の①レビュー+⑤の突き合わせが受け持つ |

詳細: 品質ゲート仕様

## 運用の全体像

人の仕事は 4 つだけ — トリガを引く・質問に答える・承認する・裁定する。この 4 つはすべてチャットからでもブラウザからでも行える。

運転モードは 5 経路あり、規模と「どこまで自動で進むか」で選ぶ。

表 4

| 経路         | 起動                              | 進む範囲             | 向く規模       |
|------------|---------------------------------|------------------|------------|
| 対話         | チャット <code>/legacy-N-xxx</code> | 1 関数 × 1 工程      | 数件・様子を見ながら |
| 会話バッチ      | チャット「まとめて N 件」                  | ①を N 件 → 一斉レビュー表 | 数十件        |
| ブラウザ単発     | 仕様書ページのボタン                      | 1 関数 × 1 工程      | 数件         |
| ブラウザ連続実行   | バッチ実行状況ページ                      | ①～⑤を工程横断で自動      | 数十～数百件     |
| 無人バッチ(CLI) | <code>pipeline.py spec</code>   | ①のみ・全件           | 数百～2000 件  |

- ・ブラウザ連続実行と無人バッチはどちらも 1 関数 = 1 headless プロセスなので、会話のトークン上限に依存しない。両者は同じ実行枠(ロック)を共有し二重起動しない
- ・進捗の閲覧と操作は WBS の HTML サイト(`serve_site.py`)。レビュアーへは **単体実行 EXE** を配布する(こちらは閲覧専用)
- ・中断・再開はいつでも安全。再開手順は常に同じ 3 コマンド(summary → next → review all)
- ・2000 関数級では WBS が自動でダッシュボード+ファイル別明細に分割される

詳細: 運用設計 / 画面設計

表 5

| 知りたいこと                                   | 読む章          |
|------------------------------------------|--------------|
| なぜこの構造か・設計判断の理由                          | アーキテクチャと設計判断 |
| 各フェーズの入出力・完了条件・情報遮断                      | パイプライン仕様     |
| functions.json / ledger.json / フロントマターの形 | データ仕様        |
| 機械レビュー・hook・スタブ検知が何を保証するか                | 品質ゲート仕様      |
| どの画面に何が出るか・HTTP API・承認 UI の設計            | 画面設計         |
| MCP ツール 30 個の入出力仕様                       | MCP サーバ仕様    |
| 導入・日常運用・無人バッチ・配布・トラブル対処                  | 運用設計         |

## 1 既存ドキュメントとの関係

表 1-1

| 文書                                         | 対象読者             | 本サイトとの関係                                      |
|--------------------------------------------|------------------|-----------------------------------------------|
| slides/index.html                          | 初見の操作者           | 操作のチュートリアル(①→⑦体験)                             |
| QUICKREF.md                                | 作業中の操作者          | コマンド即引き 1 枚                                   |
| MANUAL.md /<br>MANUAL.html /<br>MANUAL.pdf | 操作者              | 背景と操作の意味・トラブル対処。<br>HTML は画像込みの単一ファイルで<br>配れる |
| 本サイト                                       | 設計者・保守者・運用管理者    | 構造・仕様・運用設計の正を集約<br>(DESIGN.md を包含・拡張)         |
| references/<br>workflow.md /<br>schema.md  | フェーズ skill・スクリプト | 実行時規則とデータの正(機械が従う<br>原本)                      |